JDBC dan Koneksi Java



D

I S

U

S

U

N

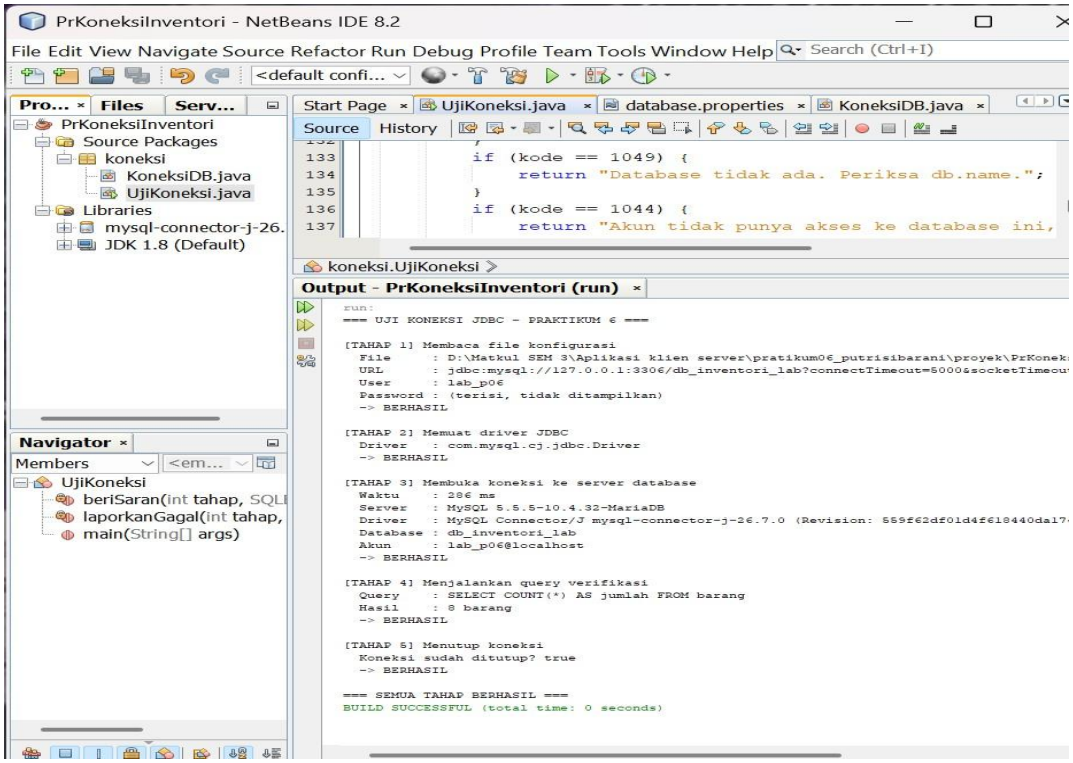
Oleh :
Nama :Nathasya uliana simbolon
Nim : 25110154

Matkul : Pemrograman Aplikasi Client Server

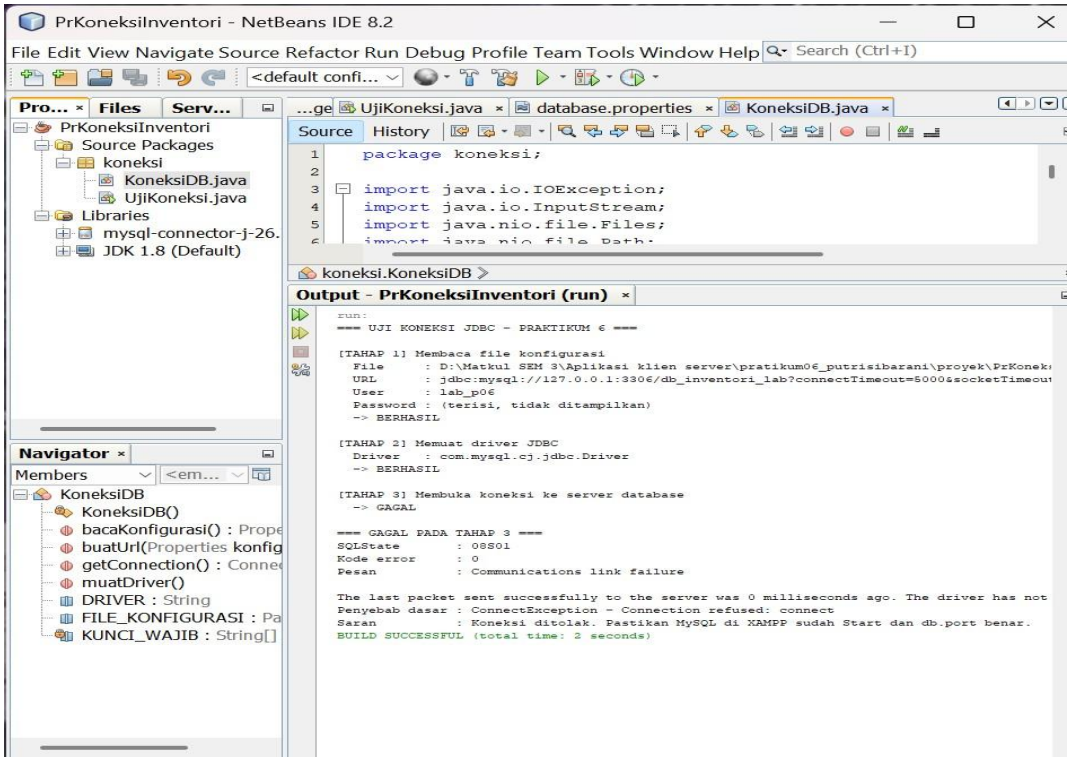
Politeknik Bisnis Indonesia 2026/2027

JDBC DAN KONEKSI JAVA

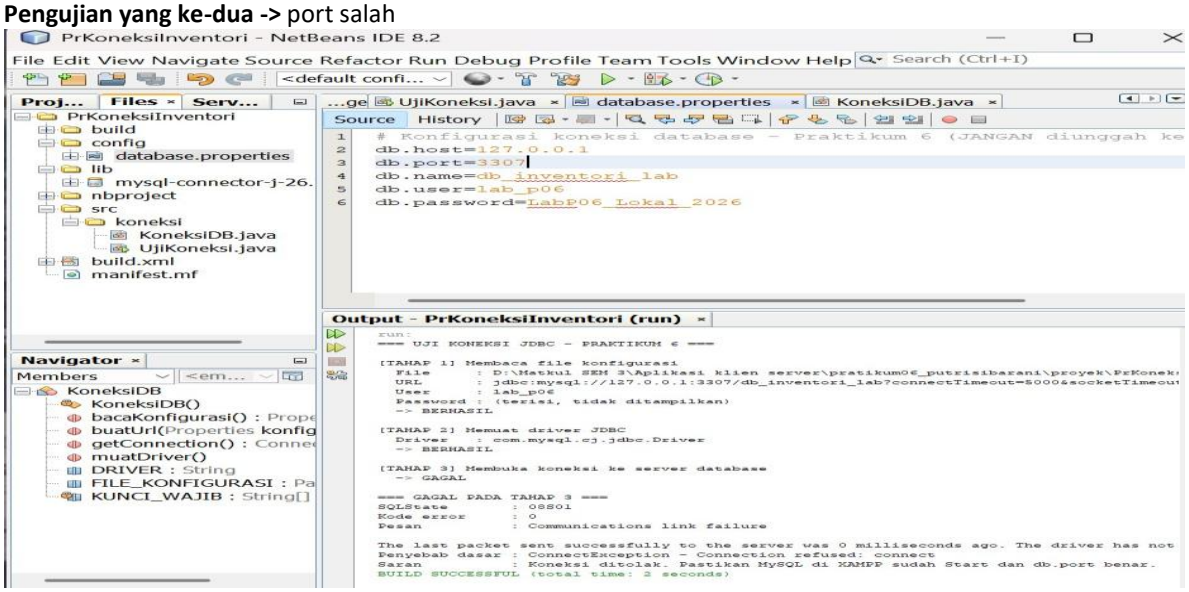
1. Hasil output pengujian di 5 tahap yang berhasil



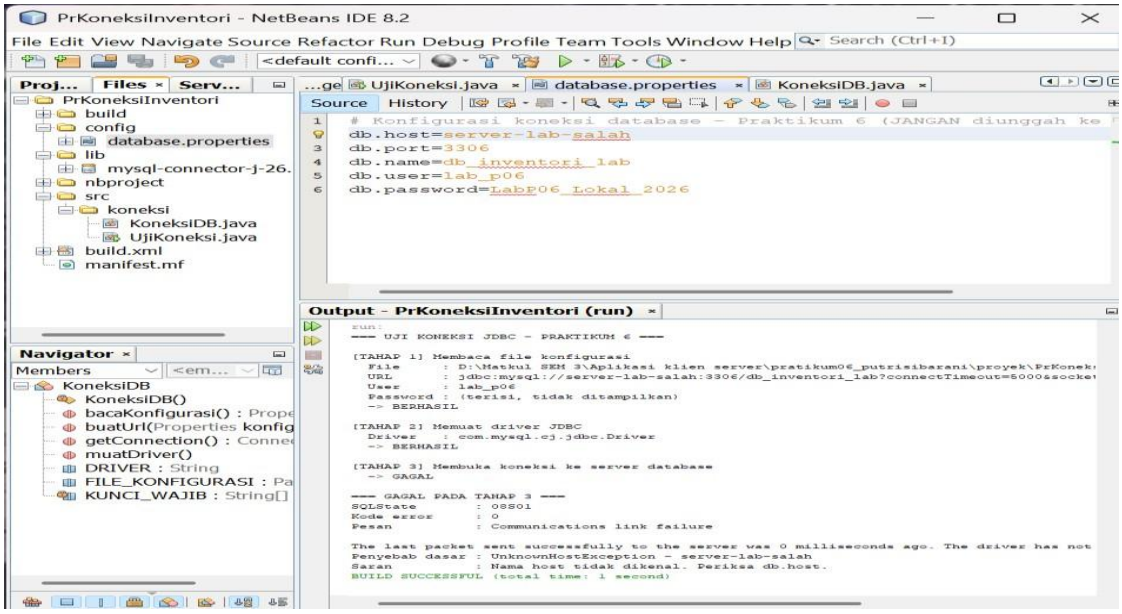
2. **Pengujian 1 >** Layanan MySQL Mati



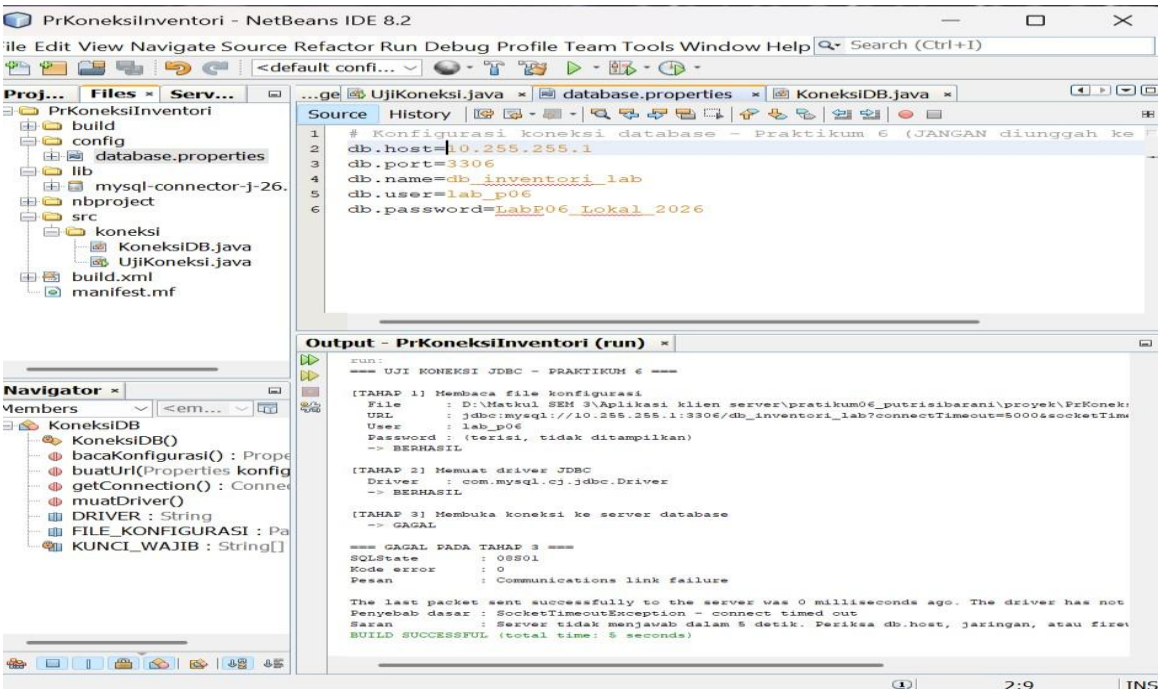
3. **Pengujian yang ke-dua ->** port salah



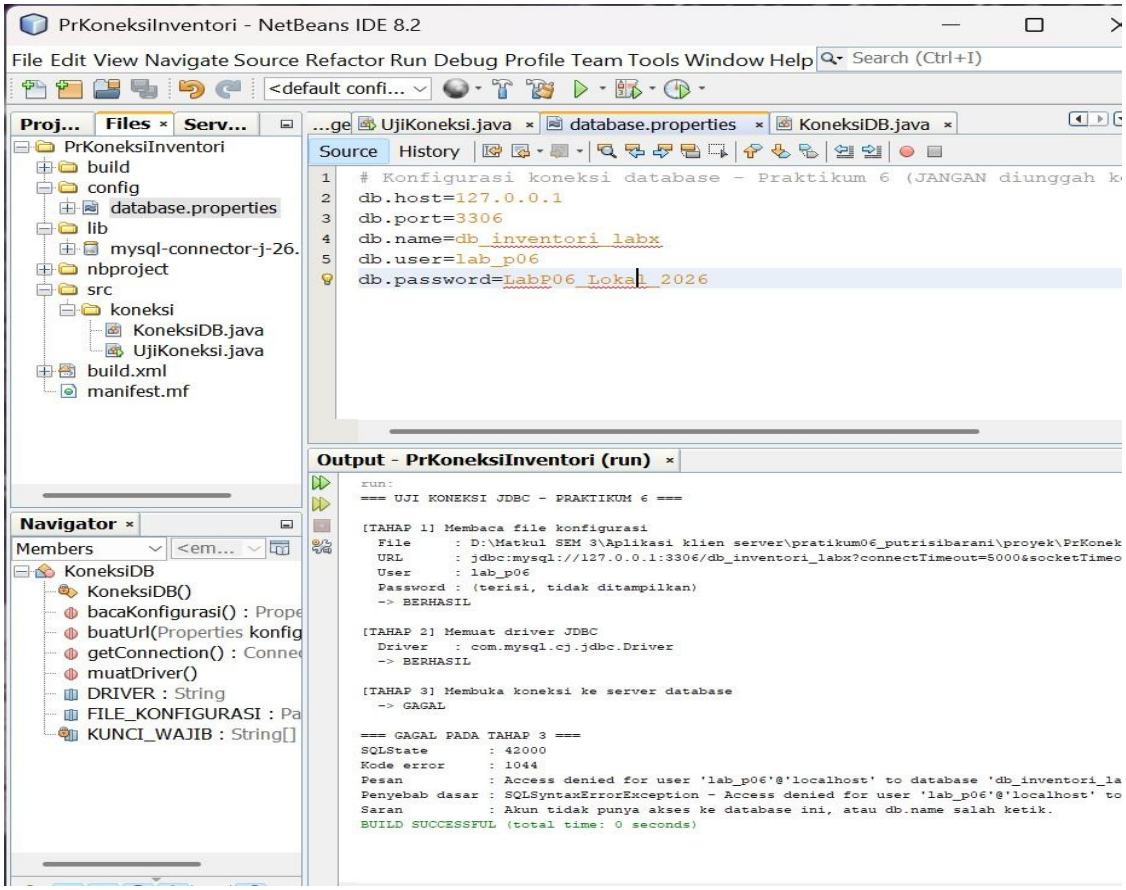
4. Pengujian ke 3 > nama host salah



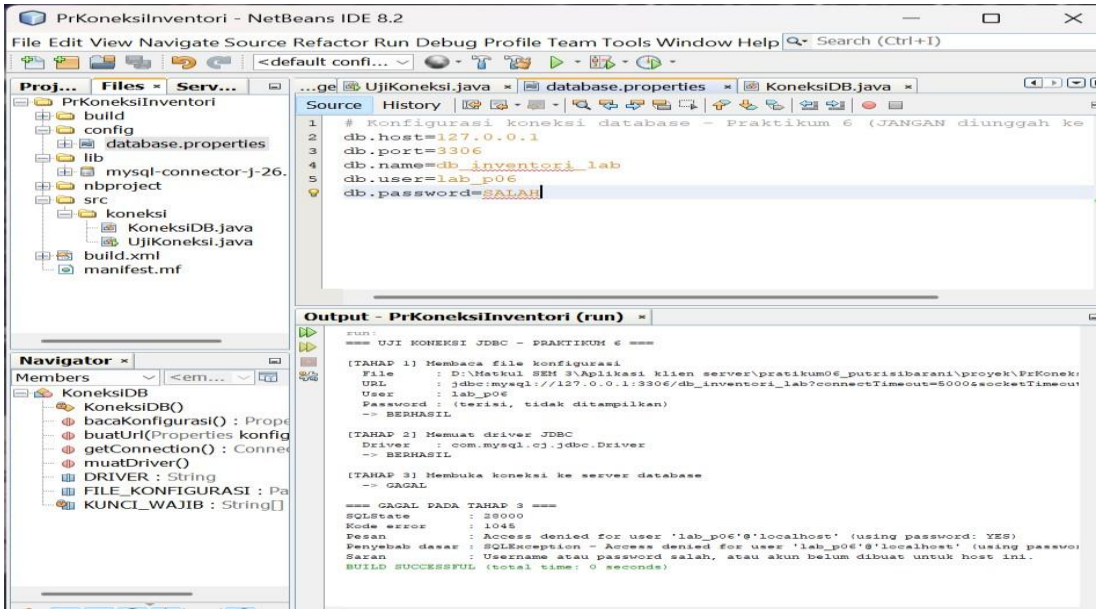
5. **Pengujian ke-4) ->** host tidak menjawab



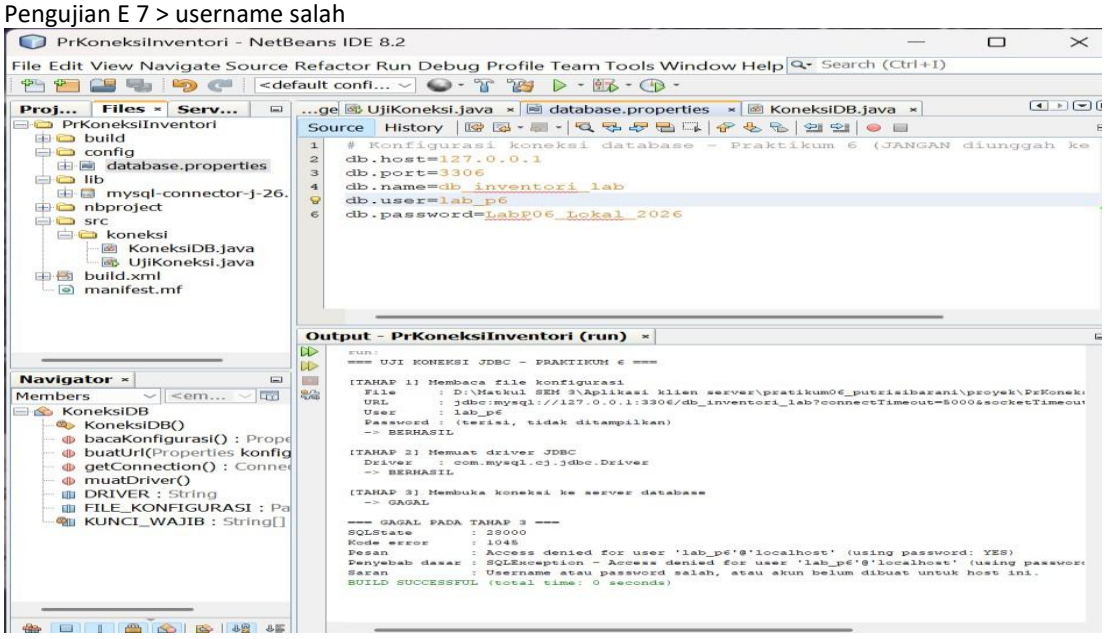
6. **Pengujian ke 5 :**nama database salah



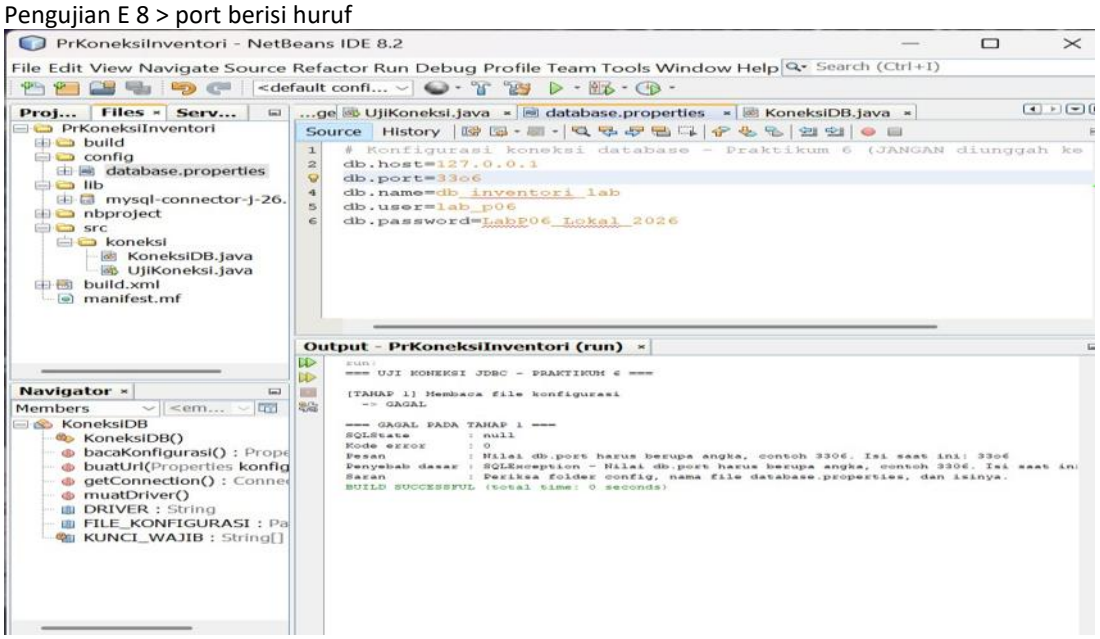
7. **Pengujian 6) ->** password salah



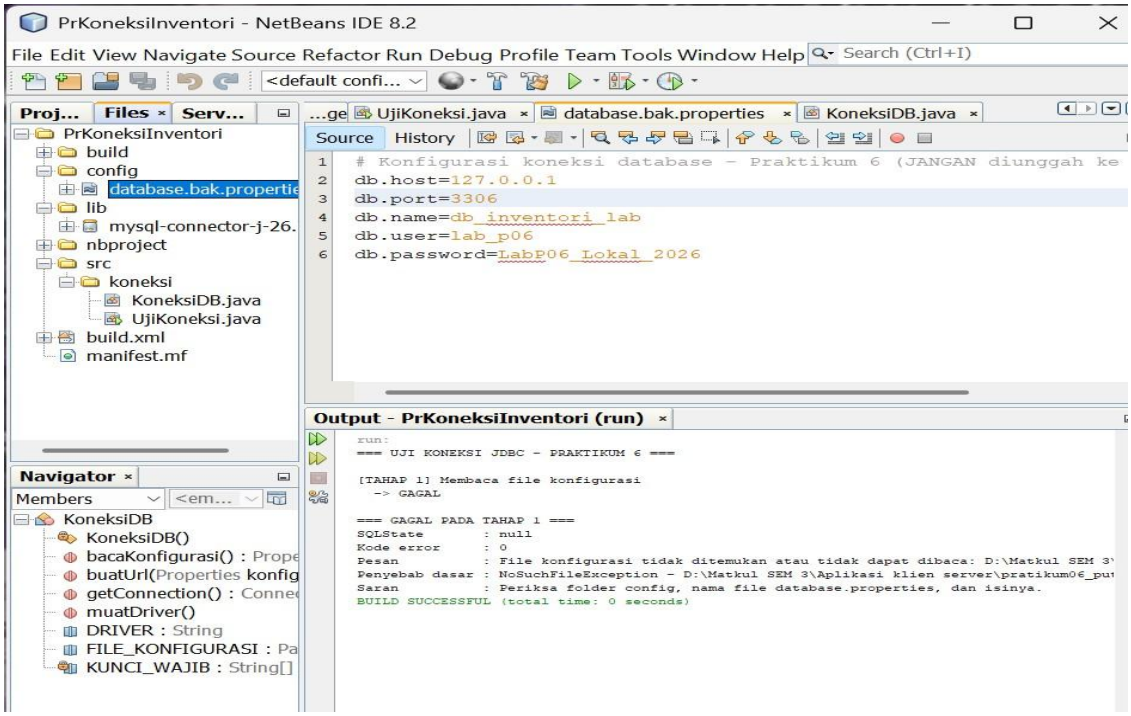
8. Pengujian E 7 > username salah



9. Pengujian E 8 > port berisi huruf



10. Pengujian -E 9) > file konfigurasi hilang



1. Apa perbedaan peran java.sql dan file mysql-connector-j-26.7.0.jari?

Jawaban :

- **java.sql**: Merupakan pustaka/paket standar (*built-in API*) yang disediakan langsung oleh Java Standard Edition (SE). Isinya berupa antarmuka (interface) dan kelas umum—seperti Connection, Statement, dan ResultSet—yang menentukan *standar cara berinteraksi* dengan basis data secara umum.
- **mysql-connector-j-26.7.0.jar**: Merupakan pustaka pihak ketiga (*external driver library*) spesifik yang dibuat oleh MySQL. File ini berisi implementasi konkret dari antarmuka java.sql agar aplikasi Java benar-benar dapat berkomunikasi secara langsung menggunakan protokol khas MySQL/MariaDB.

2. Sebutkan bagian-bagian URL JDBC yang Anda pakai beserta artinya.?

   URL: jdbc:mysql://127.0.0.1:3306/db_inventori_lab?connectTimeout=5000&socketTimeout=10000

Jawaban :

- ✓ **dbc:mysql://**: Menunjukkan protokol bahwa koneksi menggunakan JDBC dengan *driver* MySQL.
- ✓ **127.0.0.1**: Alamat IP host tempat server database berada (localhost / komputer lokal).
- ✓ **3306**: Nomor port yang digunakan oleh layanan server MySQL.
- ✓ **db_inventori_lab**: Nama database spesifik yang akan diakses.
- ✓ **connectTimeout=5000**: Batas waktu maksimal (dalam milidetik / 5 detik) untuk mencoba membuat koneksi

awal ke database sebelum *timeout*.

- ✓ **socketTimeout=10000**: Batas waktu maksimal (10 detik) menunggu respons data dari server melalui koneksi yang sudah terbentuk.

3. method apa saja yang dipanggil getConnection() sebelum DriverManager.getConnection(...), dan bagaimana urutannya?

Jawaban :

Berikut adalah method-method yang dipanggil beserta urutannya:

I. **DriverManager.getDrivers()** (atau iterasi internal pada daftar driver yang terdaftar)

   a. **Penjelasan:** DriverManager akan mengambil atau memeriksa daftar semua driver JDBC yang telah terdaftar (diregistrasikan) di dalam sistem.

II. Driver.acceptsURL(url)

   a. **Penjelasan:** DriverManager akan melakukan perulangan (*looping*) terhadap setiap driver yang ada di dalam daftar tersebut. Untuk setiap driver, method acceptsURL(String url) milik interface Driver akan dipanggil

III. Driver.connect(url, info)

   a. **Penjelasan:** Setelah menemukan driver yang mengembalikan nilai true pada method acceptsURL, DriverManager akan langsung memanggil method connect(...) dari driver spesifik tersebut untuk membuat koneksi riil ke database.

4. E1 dan E2 menghasilkan pesan yang sama. Bagaimana cara memastikan penyebab sebenarnya?

Jawaban :

Pesan error seperti *Communications link failure* bersifat umum ketika aplikasi gagal terhubung ke database. Untuk memastikan penyebab pastinya:

1) Periksa panel kontrol XAMPP untuk memastikan apakah layanan **MySQL sedang aktif (Start)** atau mati total (penyebab E1).
2) Periksa apakah ada perubahan konfigurasi port (misalnya database beralih ke port 3307) atau kesalahan penulisan port pada string koneksi Anda (penyebab E2).
3) Periksa file log error MySQL untuk melihat detail kendala di sisi server.

5. Mengapa server tidak membedakan "username salah" dari "password salah"?

Jawaban :

Demi alasan keamanan (security). Jika server memberikan pesan yang berbeda (misalnya: "Username tidak ditemukan" vs "Password salah"), peretas atau pihak yang tidak berwenang dapat memanfaatkan informasi tersebut untuk menebak-nebak akun valid.

6. Apa bukti bahwa try-with-resources sudah menutup koneksi? Apa akibatnya jika koneksi tidak pernah ditutup?

Jawaban :

1. Bukti: Program dapat berjalan lancar dan mengeksekusi blok kode hingga selesai, di mana secara

otomatis struktur di dalam blok try(...) yang mengimplementasikan AutoCloseable (seperti objek Connection, Statement, dll) akan langsung memanggil metode .close() begitu blok kode selesai dijalankan.

2. Akibat jika tidak ditutup: Sumber daya database (database resources) akan terus terbuka dan menumpuk, menyebabkan jumlah kuota koneksi aktif di server semakin berkurang (connection leak). Jika dibiarkan..

7. Mengapa password disimpan di database.properties dan tidak ditulis di kode Java?

Jawaban :

Untuk menerapkan prinsip pemisahan konfigurasi (separation of concerns) dan keamanan kode. Jika password ditulis secara langsung (hardcoded) di dalam file kode sumber Java, maka kredensial tersebut akan ikut terekspos jika kode program diunggah ke repositori publik (seperti GitHub) atau dibagikan ke orang lain. Dengan menggunakan file database.properties, data sensitif dapat dipisahkan dan dikecualikan dari version control.